Detection of Deepfakes and AI-Generated Images: A Review of Methods and a Small-Scale Empirical Study of Generalization to Unseen Generators

Adema Akhmetova* and Mansur Murzhanov*

* Astana IT University, Astana, Kazakhstan

242177@astanait.edu.kz, 241462@astanait.edu.kz

Abstract

Images produced by generative models are now easy to make and hard to tell from real photographs. Many detectors reach almost perfect accuracy on the generators they were trained on, but their accuracy often drops when a new generator appears. This paper has two parts. First, we review the main families of detection methods (artifact-based CNN detectors, frequency-based methods, detectors built on large pre-trained vision models and diffusion-specific methods) and the datasets used to evaluate them, with a focus on the generalization problem. Second, we report a small-scale empirical study of this problem. We build a dataset of 5,000 images: 3,000 images from the Community Forensics dataset (1,500 real and 1,500 generated by 12 randomly selected generators) and 2,000 images that we collected ourselves (1,000 real photographs from COCO and 1,000 images generated or partially re-synthesized with SDXL-Turbo). We train three detectors that are often used as baselines: a small CNN trained from scratch, a fine-tuned ResNet-50 and a logistic regression on frozen CLIP ViT-L/14 features. Each detector is tested at three distances from the training data: on new images from the nine training generators, on three held-out generators and on our own set, which uses a different source of real photographs and a modern diffusion generator. Near-duplicate images are kept on one side of every split boundary, networks are trained with three seeds and all metrics are reported with 95% bootstrap confidence intervals. On seen generators the two pre-trained detectors reach an AUC of 0.98–1.00. On the held-out generators the CLIP-based detector keeps an AUC of 1.00, while ResNet-50 drops to 0.90, and the drop is concentrated on the two non-GAN generators. On our own set, accuracy at the fixed threshold of 0.5 falls to 0.57–0.70 for all three detectors, but the ranking information is not lost in the same way: the CLIP-based detector keeps an AUC of 0.85, and a threshold chosen on 200 images from the new domain recovers an accuracy of 0.77. Image-to-image samples remain the hardest case for every detector. We also discuss practical pitfalls that can make detection results look better than they are: differences in file format and resolution between real and fake images, and near-duplicate images that cross the split boundaries. Code, split files and all predictions are available at https://github.com/a33558771-prog/deepfake-generalization.

Key Words: AI-generated image detection, deepfake detection, generalization, generative adversarial networks, diffusion models, CLIP, Community Forensics, decision threshold calibration, confidence intervals, near-duplicate leakage.

1 Introduction

Generative models have changed how easy it is to create a realistic image. A few years ago a convincing photograph of a person who does not exist required a research team and a GPU cluster; today anyone can type a sentence into a free web service and receive a photorealistic image in seconds, or “enhance” a real photograph until most of its pixels are no longer the ones the camera recorded. Such images are called deepfakes when they involve faces and AI-generated images in general. They are used for art and entertainment, but also for fraud, misinformation and harassment, which is why automatic detection of generated images has become an active research area [1, 2].

The detection task looks simple: given an image, decide whether it was recorded by a camera or produced by a generative model. A standard convolutional neural network (CNN) trained on one generator reaches almost 100% accuracy on new images from the same generator [3]. The problem starts when the generator changes: a detector trained on GAN images may fail on a diffusion model, and a detector trained on last year’s model may fail on this year’s [4, 5]. Because new generators appear every month, a detector that only works on known generators is of limited practical value. This generalization problem is the main topic of this paper.

Our contribution has two parts. The first part is a review. We summarize the main families of detection methods, the datasets that are used to evaluate them, and what is known about why detectors fail to generalize. We write this review from the point of view of students who want to understand the field and choose a reasonable baseline, so we focus on the methods that are most often used as references rather than on the latest leaderboard results.

The second part is a small-scale empirical study. We build a dataset of 5,000 images that combines a subset of the Community Forensics dataset [6], a recent large-scale collection of images from thousands of generators, with 2,000 images that we collected ourselves. Our own part uses a different source of real photographs (COCO [7]) and a modern distilled diffusion generator (SDXL-Turbo [8]) that is not present in the Community Forensics subset we use. On this data we train three detectors that are often used as baselines and test each of them at three distances from the training data: on new images from the training generators, on generators that were held out from training, and on our own set. The difference between the three settings is a direct measure of how much each detector loses as the test data moves away from the training data. The study is small: twelve generators, three of them held out, one external generator, and about 1,800 training images. We therefore do not try to rank detector families in general. Our aim is to measure the three-level drop carefully, with repeated runs and confidence intervals, and to separate two things that are often mixed in such results: a detector that no longer orders real and generated images correctly, and a detector whose decision threshold is simply in the wrong place for new data. We also pay attention to three technical details that are easy to overlook and that can make a detector look better than it really is: the file format and the resolution of real and fake images, and near-duplicate images that fall on both sides of a split.

The rest of the paper is organized as follows. Section 2 reviews detection methods, benchmarks and the generalization problem. Section 3 describes the dataset, Section 4 the detectors and the evaluation protocol, Section 5 the results. Section 6 discusses them and Section 7 concludes.

2 Related Work

2.1 Deepfakes and AI-generated images

The term deepfake first meant face-swap videos, and the first large benchmarks (FaceForensics++ [1], DFDC [9], Celeb-DF [10]) were built around face manipulation. In parallel, generative adversarial networks (GANs) [11] learned to generate whole images: StyleGAN and its successors [12, 13] produce faces that people cannot tell from photographs, and class-conditional models such as BigGAN [14] did the same for natural images. Since 2021, diffusion models [15, 16] and the text-to-image systems built on them (GLIDE [17], Stable Diffusion [16], SDXL [18]) have become the dominant family; distilled versions such as SDXL-Turbo [8] generate a photorealistic image in a few steps on a consumer GPU. The detection problem has therefore moved from “find the face swap” to “decide whether any image, of any content, was generated by any of thousands of models”.

2.2 Families of detection methods

Artifact-based CNN detectors train a binary classifier directly on real and generated images. Rössler et al. [1] detected face manipulations with an XceptionNet, and Wang et al. [3] showed that a ResNet-50 trained on a single GAN with blur and JPEG augmentation transfers to other GANs of the same era, because CNN generators share low-level up-sampling artifacts [19]. The weakness follows from the strength: when the generator architecture changes, the artifacts change too. **Frequency-based methods** exploit the periodic patterns that up-sampling leaves in the Fourier or DCT spectrum [20, 21, 22]; they are cheap and interpretable but sensitive to resizing and compression, and the patterns differ between generator families. **Detectors built on large pre-trained models** follow Ojha et al. [4], who kept the image encoder of CLIP ViT-L/14 [23] frozen and trained only a nearest-neighbour or linear classifier on top; this “universal fake detector” generalizes across GAN and diffusion generators although CLIP was never trained for forensics, and Cozzolino et al. [24] confirmed that it needs few training images and tolerates compression. Frozen CLIP features with a linear classifier are now a standard strong baseline and one of our three detectors. **Diffusion-specific methods** start from the finding of Corvi et al. [5] that GAN artifacts do not transfer to diffusion images: DIRE [25] uses the reconstruction error of a pre-trained diffusion model as the detection signal, and NPR [26] uses neighbouring-pixel relationships as a generator-independent artifact. They generalize better within the diffusion family but are heavier and depend on assumptions about the generator.

2.3 Benchmarks and the generalization problem

What “generalization” means in a paper is decided by its evaluation data. ForenSynths [3], with 11 CNN-based generators, was the GAN-era standard; GenImage [27] pairs ImageNet classes with eight mostly diffusion generators; Community Forensics [6], released in 2025 by Park and Owens, is the largest collection to date,

about 2.7 million images from 4,803 generators, and its authors show that training on many generators improves generalization more than more images from a few. Yan et al. [28] built the Chameleon set from generated images that human annotators judged to be real and found most published detectors close to chance on it, a warning that benchmark numbers do not guarantee performance on images people meet online. Across these works the same pattern appears: near-perfect accuracy on the training generator, lower accuracy on the same family, and a large drop on a new family or after social-media compression. Chai et al. [29] showed that detectors rely on local low-level patches rather than global semantics, and Gragnaniello et al. [30] found that most published gains disappear under realistic resizing and compression. Two practical problems are discussed less often: real and fake images in a benchmark often come from different pipelines (generated images saved as PNG at a fixed resolution, real photographs as JPEG files of varying size), and their content may differ, so a classifier can learn the pipeline or the topic instead of the artifact. Our experiment controls both and tests generalization at three levels: new images, new generators and a new data source.

3 Dataset

Our dataset has 5,000 images in two parts: 3,000 images from Community Forensics [6] and 2,000 images that we collected ourselves. Both parts share one file format and one metadata table, so that any experiment can be run on any combination of them. Figure 1 shows random examples.

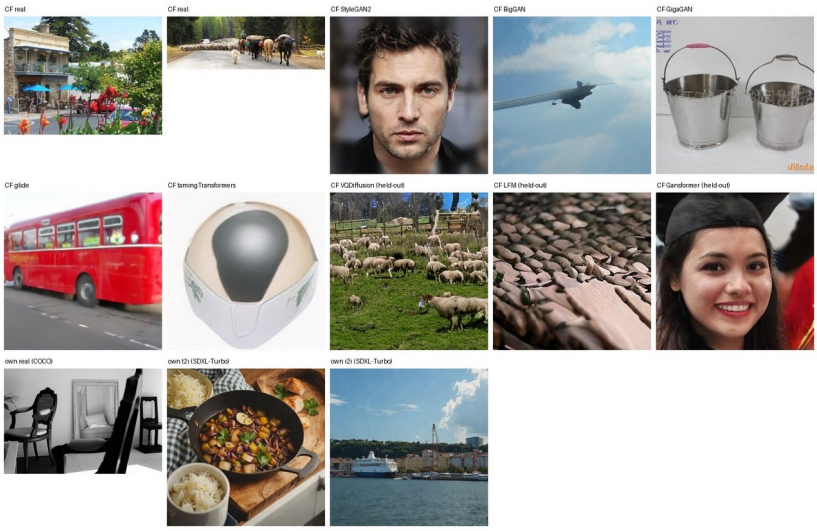


Figure 1: Random examples from the dataset. Top and middle rows: real images and generated images from Community Forensics (nine generators used for training, three held-out generators). Bottom row: our own set — a real COCO photograph, an image generated by SDXL-Turbo from a COCO caption, and a real COCO photograph partially re-synthesized by SDXL-Turbo (image-to-image, strength 0.5).

3.1 Community Forensics subset

Community Forensics is distributed as parquet files of 1–13 GB each, which is far beyond what we could process on a free Google Colab session. We therefore used the Community Forensics-Small release, which the authors recommend for experiments and which includes real images with redistributable licenses, and we read only files smaller than 1.5 GB. The selection was done in five steps, all with fixed seeds, in notebook 02 of the repository. (1) From the parquet files smaller than 1.5 GB we read the metadata of every image (label, generator name, prompt) without decoding the images. (2) We listed the generators that had at least 125 generated images in the files read; this list is the population from which we sampled. (3) From this list we drew 12 generators with Python's random.sample and seed 42. (4) From these 12 we drew 3 held-out generators with random.sample again; they never appear in training or validation. (5) Within each of the 12 generators we drew 125 images, and from the real part of the release 1,500 images, with DataFrame.sample(random_state=42). The list of candidate generators, the drawn identifiers and the seed are stored in the repository, so the subset can be rebuilt exactly. Table 1 lists the generators. Because small parquet files mostly belong to models from the Manual subset of Community Forensics (models the authors downloaded and ran by hand), the candidate list and therefore our subset is dominated by GANs; it also contains an autoregressive model (Taming

Transformers), a text-conditioned diffusion model (GLIDE), a discrete diffusion model (VQ-Diffusion) and a latent flow-matching model (LFM). We return to this bias in Section 6.

Table 1: Generators in the Community Forensics part of our dataset (125 images each).

Generator	Family	Reference	Role
BigGAN	GAN (class-conditional)	Brock et al. 2019 [14]	training
GigaGAN	GAN (text-conditional)	Kang et al. 2023 [31]	training
Projected GAN	GAN	Sauer et al. 2021 [32]	training
StyleGAN2	GAN	Karras et al. 2020 [12]	training
StyleGAN3	GAN	Karras et al. 2021 [13]	training
StyleGAN-XL	GAN	Sauer et al. 2022 [33]	training
StyleSAN-XL	GAN	Takida et al. 2024 [34]	training
GLIDE	Diffusion (text-to-image)	Nichol et al. 2022 [17]	training
Taming Transformers	Autoregressive (VQGAN)	Esser et al. 2021 [35]	training
Gansformer	GAN + attention	Hudson and Zitnick 2021 [36]	held out
LFM	Latent flow matching	Dao et al. 2023 [37]	held out
VQ-Diffusion	Discrete diffusion	Gu et al. 2022 [38]	held out

3.2 Our own set

The second part of the dataset tests the detectors on data that was not collected by the Community Forensics authors: 1,000 real photographs and 1,000 generated images. The real photographs were sampled at random from COCO val2017 [7], ordinary Flickr photographs with short human-written captions; we used COCO rather than our own phone photographs because it is public and makes the experiment reproducible.

The generated images were produced with SDXL-Turbo [8], a distilled version of SDXL that generates a 512 × 512 image in four denoising steps without classifier-free guidance (guidance scale 0). We used two procedures. For 600 text-to-image samples, we took the first human caption of a random real COCO image, appended the words “, realistic photo”, and generated an image from this prompt with 4 steps. Because the prompts come from the captions of our real images, the real and generated parts have the same distribution of topics (food, people, animals, street scenes, interiors), which prevents the detector from separating real from fake by content. For 400 image-to-image samples, we took one of the 400 real COCO photographs of our set whose caption had not been used for a text-to-image prompt, resized it to 512 × 512, and passed it through the SDXL-Turbo image-to-image pipeline with its own caption plus “, realistic photo” as the prompt, guidance scale 0 and strength 0.5. With 4 scheduled steps, a strength of 0.5 means that the input is encoded to the latent space, noised to the middle of the noise schedule, and only the last 2 of the 4 denoising steps are run from that noised latent. The result keeps the layout, the colours and often fine texture of the original photograph, while the model redraws details; we call these images partially re-synthesized (Figure 1, bottom row). This simulates “AI-enhanced” photographs, which are common on social media and are harder to detect than fully generated images. For every image-to-image sample we recorded the identifier of the source photograph (original_id in the metadata), so that the link between an edited image and its source is never lost. This also means that our set is paired: 400 of the 1,000 real photographs have a partially re-synthesized version with the opposite label in the same test set, and the other 600 share their caption with a text-to-image sample. We keep this design on purpose, because it asks the hardest question, whether a detector can separate a photograph from its own edit, and because it keeps the content of the two classes identical. But it also means that the real and generated parts of our set are not independent samples, so in Section 5 we report, next to the full numbers, the results with the 400 source photographs removed and a paired comparison of each edited image with its source. Nothing from our set is used in training or validation.

3.3 Preprocessing and controls

All 5,000 images were converted to the same format before any experiment: RGB, JPEG with quality 95, at most 512 pixels on the longer side. Generators output PNG files and cameras output JPEG files, and a classifier

can separate PNG from JPEG with very high accuracy without looking at the content; after the conversion all images share one compression pipeline.

Resolution is a second source of leakage: the generated images in Community Forensics are mostly 256 × 256 pixels, the real images are larger. Every image is therefore resized so that its shorter side is 256 pixels and a 224 × 224 crop is taken (random in training, central at test), so real and generated images pass through the same resizing and the detector cannot use image size as a shortcut.

3.4 Splits

The Community Forensics part was divided into training, validation and test sets within each generator and within the real images (70% / 15% / 15%), so that all three sets have the same composition. The three held-out generators form a separate test set of unseen generators, and our own set is used only for testing.

A random split by image is not enough when a dataset contains near-duplicates. Several generators in Community Forensics-Small, in particular StyleGAN2 and StyleGAN3, produce many almost identical faces, and a detector that sees one of them in training is helped when a near-copy appears in the test set. We checked all 5,000 images with a 64-bit difference hash (dHash) and counted pairs of images of the same generator or real source at Hamming distance 8 or less. In the Community Forensics part this gave 662 such pairs (4 of them exact duplicates), involving 118 images, 70 from StyleGAN2, 35 from StyleGAN3, 7 from Gansformer and a few real images. In our first splits, 313 of these pairs crossed a split boundary. We therefore rebuilt the splits at the level of clusters: images of the same generator or real source at Hamming distance 8 or less were joined into clusters (2,898 clusters for 3,000 images; the largest has 60 images), each cluster was assigned to a single split, and the 70 / 15 / 15 proportion was kept per generator as closely as the cluster sizes allow. After this step no near-duplicate pair of the same generator crosses a boundary (0 pairs), and 1,237 images changed split. (118 pairs of images from different generators or sources still fall at distance 8 or less across splits; these are visually similar but independently generated images, for example two StyleGAN faces from different models, and we did not join them because they carry different labels or generator identities.) Our own set is used only for testing, so none of its images or their sources is seen in training; the 400 real/edited pairs inside it are described in Section 3.2 and handled separately in Section 5. All results in Section 5 use the new splits; the dedup report and the split files are in the repository. Table 2 summarizes the splits.

Table 2: Data splits after cluster-level de-duplication. The unseen-generator test set contains only generated images; for metrics that need both classes it is combined with the 225 real images of the seen-generator test set.

Split	Source	Real	Fake	Total	Purpose
train	CF, 9 generators	1,050	786	1,836	training
val	CF, 9 generators	225	170	395	model selection, threshold
test-seen	CF, 9 generators	225	169	394	seen generators
test-unseen	CF, 3 held-out generators	–	375	375	unseen generators
test-own	COCO + SDXL-Turbo	1,000	1,000 (600 t2i, 400 i2i)	2,000	new source and generator

4 Experimental Setup

4.1 Detectors

We compare three detectors that are often used as baselines in the literature and that differ in how much prior knowledge they bring. All three output a single score, the probability that the image is generated, and are trained on the same training split.

SimpleCNN. A small convolutional network trained from scratch: four blocks of 3×3 convolution, batch normalization, ReLU and 2×2 max-pooling with 32, 64, 128 and 256 channels, followed by global average pooling, dropout (0.3) and a linear layer. It has about 0.4 million parameters. This is the baseline: a detector with no prior knowledge about images.

ResNet-50. A ResNet-50 [39] pre-trained on ImageNet, with the final layer replaced by a single output and all layers fine-tuned. This is the standard architecture of artifact-based detectors such as [3].

CLIP + linear. The image encoder of CLIP ViT-L/14 [23], kept frozen, followed by a logistic regression classifier on the standardized, L2-normalized 768-dimensional image embeddings, in the spirit of the universal detector of Ojha et al. [4]. The regularization parameter C was chosen on the validation set among {0.01, 0.1, 1, 10}; C = 0.1 gave the best validation AUC (0.998). This detector has no random component once the features are fixed, so it is fitted once.

4.2 Training details

SimpleCNN was trained for 10 epochs with AdamW (learning rate 10^{-3}, weight decay 10^{-4}) and ResNet-50 for 5 epochs with AdamW (learning rate 10^{-4}, weight decay 10^{-4}), both with a cosine learning-rate schedule, batch size 32, binary cross-entropy loss with a positive-class weight equal to the real/fake ratio of the training set, and random 224 × 224 crops and horizontal flips as augmentation. Each network was trained three times with seeds 0, 1 and 2, which control the initialization of the new layers and the order of the training data; for each run the epoch with the best validation AUC was kept: epoch 10 for SimpleCNN (validation AUC 0.774–0.802 over the three seeds), epoch 5 for ResNet-50 (validation AUC 0.971–0.979 over the three seeds). CLIP features were extracted once for all images after the same resizing to 256 pixels. The revised experiments were run on an Apple M3 laptop GPU (PyTorch MPS backend); one epoch took about 30 s for SimpleCNN and 2 min for ResNet-50, and the whole protocol (three seeds, CLIP features, bootstrap) about 50 minutes. The code, the split files, the configuration actually used (config.json) and the predictions of every detector on every test image are available at https://github.com/a33558771-prog/deepfake-generalization.

4.3 Evaluation protocol

Each detector is evaluated on three test sets of increasing difficulty: (1) test-seen, new images from the nine training generators; (2) test-unseen, images from the three held-out generators, combined with the real images of test-seen so that both classes are present; (3) test-own, our COCO + SDXL-Turbo set, which differs from the training data in both the real source and the generator. We report accuracy at a fixed threshold of 0.5, the area under the ROC curve (AUC), which does not depend on a threshold, and average precision (AP). We also report the detection rate, that is the fraction of generated images classified as generated at threshold 0.5, separately for every generator and for the two kinds of SDXL-Turbo images, and the false alarm rate on real images. The threshold of 0.5 is never tuned on a test set; Section 4.4 describes a separate analysis in which the threshold is varied.

4.4 Uncertainty and threshold analysis

Repeated runs. For SimpleCNN and ResNet-50 every metric is the mean over the three seeds, and we report the standard deviation over seeds. The CLIP-based detector is deterministic, so its standard deviation over seeds is zero and only the bootstrap interval below applies.

Confidence intervals. The three test sets are small (394 to 2,000 images), so a difference of a few percentage points may be noise. For every metric we report a 95% percentile bootstrap confidence interval over test images: we resample the test set with replacement 1,000 times, compute the metric for each seed on the resample, average over seeds and take the 2.5th and 97.5th percentiles. The intervals therefore reflect the size of the test set; the standard deviations reflect the training randomness.

Ranking versus threshold. An accuracy of 0.6 at threshold 0.5 can mean that the detector no longer separates real from generated images, or that it separates them but its scores have shifted so that 0.5 is no longer the right cut; AUC measures only the first. To separate the two on our own set we report accuracy at four thresholds: (a) the fixed 0.5 used everywhere else; (b) the threshold that maximizes balanced accuracy on the Community Forensics validation split, which a practitioner could choose without any access to the new domain; (c) the threshold that maximizes balanced accuracy on 200 stratified images drawn from our own set (100 real, 100 generated), evaluated on the remaining 1,800 images, repeated 20 times; (d) the oracle threshold that maximizes balanced accuracy on the whole set, which is not achievable in practice and serves as an upper bound. We also report the median score of real and generated images on our own set.

Paired evaluation. Because 400 real photographs of our own set have an edited version in the same set (Section 3.2), we report three additional numbers for test-own: the accuracy and AUC with the 400 source

photographs removed (600 real, 1,000 generated); the AUC of the text-to-image samples against the 600 photographs whose captions they were generated from, and of the image-to-image samples against their 400 source photographs; and the paired detection rate, the fraction of the 400 pairs in which the edited image receives a higher generated-score than its own source photograph (0.5 means the detector cannot tell a photograph from its edit). These are computed from the saved predictions with paired_analysis.py in the repository.

5 Results

Table 3 gives the main results and Figure 2 shows them as bar charts. Table 4 gives the detection rate for every generator and the false alarm rate for every real-image source. For the two networks every number is the mean over three seeds, written as mean ± standard deviation; brackets give the 95% bootstrap confidence interval over test images (Section 4.4). Table 5 and Figure 5 give the threshold analysis on our own set.

Table 3: Main results. Accuracy at threshold 0.5, AUC and AP of the three detectors on the three test sets, mean ± std over three seeds (the CLIP-based detector is deterministic), with 95% bootstrap confidence intervals over test images in brackets. test-unseen contains the 375 held-out generated images and the 225 real images of test-seen.

Detector	Test set	Accuracy @ 0.5	AUC	AP
SimpleCNN	test-seen	0.750 ± 0.003 [0.711, 0.788]	0.814 ± 0.006 [0.770, 0.856]	0.812 ± 0.006 [0.759, 0.858]
	test-unseen	0.703 ± 0.002 [0.669, 0.737]	0.791 ± 0.003 [0.756, 0.825]	0.881 ± 0.002 [0.851, 0.904]
	test-own	0.573 ± 0.011 [0.554, 0.592]	0.621 ± 0.013 [0.598, 0.645]	0.608 ± 0.012 [0.578, 0.642]
ResNet-50	test-seen	0.938 ± 0.008 [0.916, 0.959]	0.982 ± 0.002 [0.971, 0.992]	0.978 ± 0.002 [0.961, 0.990]
	test-unseen	0.821 ± 0.005 [0.792, 0.847]	0.904 ± 0.004 [0.883, 0.926]	0.950 ± 0.002 [0.936, 0.963]
	test-own	0.616 ± 0.013 [0.597, 0.634]	0.683 ± 0.020 [0.661, 0.703]	0.654 ± 0.020 [0.623, 0.683]
CLIP + linear	test-seen	0.987 [0.975, 0.997]	0.997 [0.993, 0.999]	0.996 [0.990, 0.999]
	test-unseen	0.982 [0.970, 0.992]	0.998 [0.995, 1.000]	0.999 [0.997, 1.000]
	test-own	0.698 [0.677, 0.716]	0.852 [0.836, 0.867]	0.826 [0.801, 0.853]

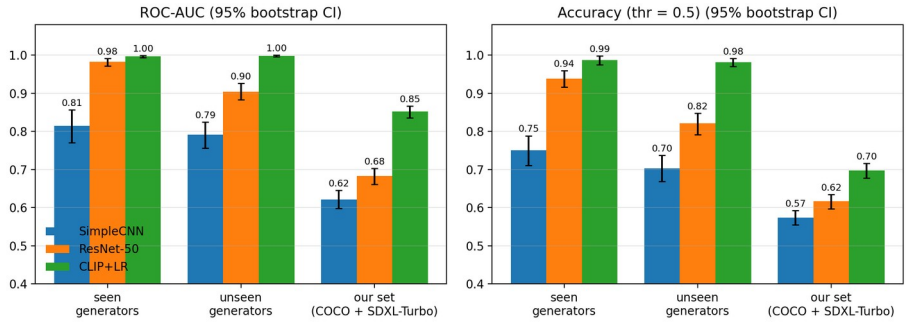


Figure 2: AUC (left) and accuracy at threshold 0.5 (right) of the three detectors on the three test sets. Bars are means over three seeds; error bars are 95% bootstrap confidence intervals over test images. All detectors lose from seen generators to unseen generators to our own set; the CLIP-based detector loses the least on unseen generators, and on our own set its AUC stays well above chance while its accuracy at the fixed threshold does not.

Table 4: Detection rate per generator (fraction of generated images classified as generated at threshold 0.5) and false alarm rate per real-image source (fraction of real images classified as generated), mean ± std over seeds. Seen generators have about 19 test images each, so their rates are coarse; held-out generators have 125 each.

Generator / source	Set	SimpleCNN	ResNet-50	CLIP + linear
BigGAN	test-seen	0.42 ± 0.05	1.00 ± 0.00	1.00
GigaGAN	test-seen	0.51 ± 0.03	0.79 ± 0.05	0.95
Projected GAN	test-seen	0.95 ± 0.00	1.00 ± 0.00	1.00
StyleGAN2	test-seen	1.00 ± 0.00	1.00 ± 0.00	1.00
StyleGAN3	test-seen	1.00 ± 0.00	1.00 ± 0.00	1.00

StyleGAN-XL	test-seen	0.47 ± 0.05	0.91 ± 0.03	1.00
StyleSAN-XL	test-seen	0.68 ± 0.05	0.95 ± 0.00	1.00
GLIDE	test-seen	0.67 ± 0.08	0.84 ± 0.05	1.00
Taming Transformers	test-seen	0.74 ± 0.05	0.91 ± 0.06	1.00
Gansformer (held out)	test-unseen	1.00 ± 0.00	1.00 ± 0.00	1.00
LFM (held out)	test-unseen	0.42 ± 0.03	0.57 ± 0.02	0.96
VQ-Diffusion (held out)	test-unseen	0.55 ± 0.02	0.68 ± 0.04	0.98
SDXL-Turbo, text-to-image (600)	test-own	0.41 ± 0.02	0.57 ± 0.02	0.51
SDXL-Turbo, image-to-image (400)	test-own	0.30 ± 0.01	0.43 ± 0.05	0.43
False alarms, CF real (225)	test-seen	0.22 ± 0.02	0.06 ± 0.01	0.02
False alarms, COCO real (1,000)	test-own	0.22 ± 0.01	0.28 ± 0.02	0.08

Seen generators. On new images from the nine training generators, the two pre-trained detectors work well: ResNet-50 reaches an accuracy of 0.938 ± 0.008 and an AUC of 0.982 ± 0.002, and the CLIP-based detector 0.987 and 0.997. The small CNN trained from scratch is clearly weaker (accuracy 0.750 ± 0.003, AUC 0.814 ± 0.006), and its confidence interval does not reach the two pre-trained detectors. Table 4 shows that it is weak in a specific way: it detects Projected GAN, StyleGAN2, StyleGAN3 well but misses most BigGAN (0.42) and StyleGAN-XL (0.47) images, and its per-generator rates vary strongly between seeds. With about 1,800 training images and no prior knowledge, the small network learned the artifacts of some generators but not a general notion of a generated image. The pre-trained detectors reach 0.84–1.00 on every seen generator except GigaGAN (0.79 and 0.95), which is the only text-conditioned GAN in the set and produces more varied images.

Unseen generators. On the three held-out generators the picture splits by family. Gansformer, a GAN with attention, is detected perfectly by all three detectors, including the small CNN; it is close to the StyleGAN-type models that dominate the training set. LFM and VQ-Diffusion, which are not GANs, are where the artifact-based detectors fail: ResNet-50 detects only 57% and 68% of them, and the small CNN 42% and 55%. The CLIP-based detector still detects 96% and 98%. In aggregate, ResNet-50 drops from an AUC of 0.982 on seen generators to 0.904 on unseen ones, while the AUC of the CLIP-based detector stays at 0.997–0.998, and its accuracy stays at 0.982; the confidence intervals of the two detectors on test-unseen do not overlap. This is the pattern reported by Ojha et al. [4]: a classifier trained end-to-end learns the artifacts of the training generators, while fixed CLIP features separate real from generated images in a way that transfers to a new family. With three held-out generators we can report this pattern, not establish it for non-GAN generators in general.

Our own set. On the COCO + SDXL-Turbo set the accuracy at the fixed threshold of 0.5 drops for all three detectors, to 0.573, 0.616 and 0.698. The AUC values tell a different story for each detector: 0.621, 0.683 and 0.852. For the small CNN and ResNet-50 the AUC is low, so these detectors have largely lost the ability to order real and generated images from the new domain. For the CLIP-based detector the AUC of 0.85 means that the ordering is still informative; its problem is the threshold. Table 5 shows this directly. The median score of real COCO photographs is 0.01 and of SDXL-Turbo images 0.47, so most generated images fall below 0.5. A threshold chosen on the Community Forensics validation split helps a little (0.732), because the shift is specific to the new domain. A threshold chosen on only 200 images of the new domain raises the accuracy of the CLIP-based detector to 0.772 ± 0.006, close to the oracle value of 0.780; for ResNet-50 and the small CNN the gain is smaller (+0.013 and +0.005). Even at the best possible threshold the CLIP-based detector stays well below its accuracy on the benchmark (0.780 against 0.982 on test-unseen), so the drop is not only a calibration problem; part of it is a real loss of separability on SDXL-Turbo images, above all on the image-to-image subset. The three detectors also fail in different ways at the fixed threshold. The CLIP-based detector stays conservative: it raises a false alarm on 8% of COCO photographs but detects only 51% of the text-to-image samples and 43% of the image-to-image samples. ResNet-50 detects more SDXL-Turbo images (57% and 43%) at the price of flagging 28% of real COCO photographs as generated. The image-to-image samples are the hardest subset for every detector: these images keep the composition, lighting and often the camera-like noise of a real photograph, and the detectors treat them as real (Figure 4, top row). The paired evaluation of Section 4.4 confirms this without the influence of the shared content: with the 400 source photographs removed, the AUC of the CLIP-based detector is 0.864 (full set: 0.852), so the pairing does not change the picture. Against their own source photographs, the text-to-image samples reach an AUC of 0.883 and the

image-to-image samples only 0.805; the edited image receives a higher score than its own source in 86% of the 400 pairs for the CLIP-based detector and 65% and 70% for ResNet-50 and the small CNN.

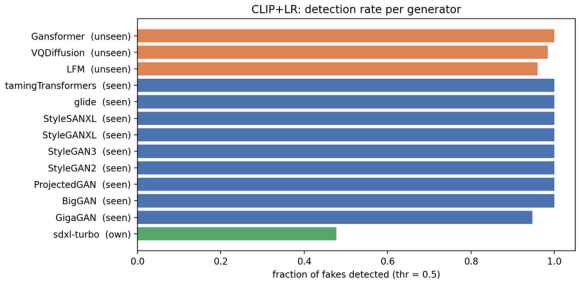


Figure 3: Detection rate per generator for the CLIP-based detector (threshold 0.5, mean over the test images; the detector is deterministic). Blue: generators seen in training; orange: held-out generators; green: SDXL-Turbo from our own set.

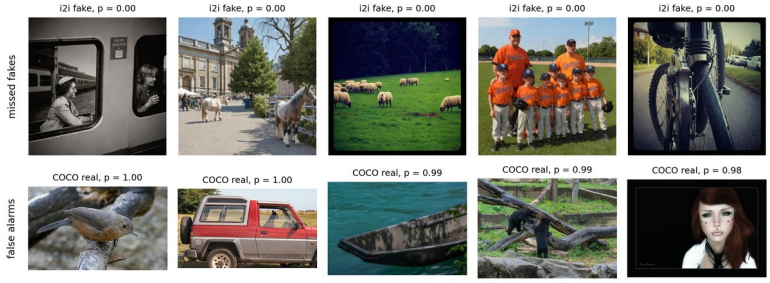


Figure 4: Errors of the CLIP-based detector on our own set. Top: image-to-image samples with the lowest generated-probability (missed fakes). Bottom: real COCO photographs with the highest generated-probability (false alarms).

False alarms. The false alarm rate on real images also changes between sources. On the real images of Community Forensics-Small, the three detectors flag 22%, 6% and 2% of photographs; on COCO photographs they flag 22%, 28% and 8%. For ResNet-50 the false alarm rate quadruples. Figure 4 (bottom row) shows typical false alarms: smooth water surfaces, repetitive textures, dark scenes and low-detail objects; one of the highest-scoring COCO “photographs” is in fact a digitally rendered portrait, a reminder that the real class of a public dataset is not perfectly clean either. Part of the drop on our own set is therefore not caused by the new generator at all, but by the new source of real photographs, whose compression history and content differ from the real images the detectors were trained on.

Table 5: Accuracy on our own set (2,000 images) at different decision thresholds, mean ± std over seeds: (a) fixed 0.5; (b) threshold chosen on the Community Forensics validation split; (c) threshold chosen on 200 images of our own set, evaluated on the other 1,800 (20 draws); (d) oracle threshold. Last two columns: median score of real and generated images.

Detector	AUC	(a) acc @ 0.5	(b) acc @ val thr.	(c) acc @ 200-image thr.	(d) acc @ oracle thr.	median score, real	median score, fake
SimpleCNN	0.621 ± 0.013	0.573 ± 0.011	0.577 ± 0.012 t = 0.49	0.579 ± 0.010	0.595 ± 0.010 t = 0.39	0.32	0.42
ResNet-50	0.683 ± 0.020	0.616 ± 0.013	0.626 ± 0.015 t = 0.42	0.629 ± 0.010	0.643 ± 0.019 t = 0.22	0.18	0.53
CLIP + linear	0.852	0.698	0.732 t = 0.35	0.772 ± 0.006	0.780 t = 0.12	0.01	0.47

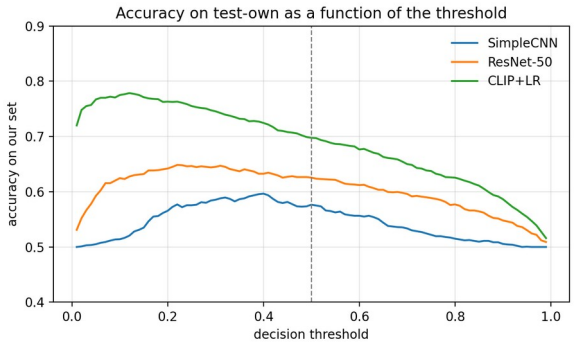


Figure 5: Accuracy on our own set as a function of the decision threshold (scores averaged over seeds for the networks). The dashed line marks the fixed threshold of 0.5 used in Tables 3 and 4. For the CLIP-based detector the best threshold lies below 0.5 (about 0.12) and gives an accuracy of about 0.78.

6 Discussion

Three levels of generalization. Our three test sets measure different things. From seen images to seen generators nothing is lost; from seen to unseen generators the loss depends on the family of the new generator (a new GAN is easy for everyone, a new non-GAN model is hard for the artifact-based detectors and easy for the CLIP-based one); from the benchmark to our own data every detector loses most of its accuracy at the fixed threshold. The ranking of the detectors is the same at every level (CLIP > ResNet-50 > SimpleCNN) and the AUC confidence intervals of the CLIP-based detector and ResNet-50 do not overlap on test-unseen or test-own, so with our test sizes this ranking is reliable for the AUC. The gap between the benchmark numbers and the numbers on our own set is in any case much larger than the gap between detectors, which matches the warning of Yan et al. [28] that benchmark accuracy is a poor predictor of performance on images people actually meet online. With three held-out generators and one external generator we cannot say how far these numbers transfer to other generators; what we can say is that the three-level drop is clearly visible even in a study of this size.

Why the CLIP-based detector generalizes better. The CLIP encoder was never trained on our data; only a logistic regression with a strong regularization ($C = 0.1$ was selected on the validation set) was fitted on top of it. The classifier therefore cannot overfit to the fine artifacts of the nine training generators, and the features it uses are the general-purpose features CLIP learned from 400 million web images. The price is visible on our own set: the detector is confident about real photographs but not about SDXL-Turbo images, because nothing in its training set looked like a modern latent-diffusion output.

Ranking versus calibration. The drop on our own set has two different causes, and Table 5 separates them. For the small CNN and ResNet-50 the AUC itself is low (0.62 and 0.68), and even the oracle threshold only brings their accuracy to 0.59 and 0.64; these detectors have mostly lost the ability to tell real COCO photographs from SDXL-Turbo images, so their problem is ranking. For the CLIP-based detector the AUC of 0.85 shows that the ordering is largely intact, but its scores for generated images have moved below 0.5 (median 0.47), so its problem at the fixed threshold is mainly calibration. A threshold chosen on only 200 images of the new domain recovers an accuracy of 0.77, against 0.70 at 0.5. The practical point is that a detector deployed on a new source of images should have its threshold checked on a small labelled sample from that source, and that accuracy at a fixed 0.5 threshold on a new domain should not be read as the detector's ability to separate the classes, nor described as “chance level” when the AUC says otherwise. We did not fine-tune any detector on our own set, because the question of the study is what happens without adaptation.

Why edited photographs are the hardest case. Our image-to-image samples were produced from real COCO photographs with a strength of 0.5, that is, only the last 2 of 4 denoising steps were run from the noised latent of the photograph, so the result keeps the global structure, colours and much of the texture of the original and is only partially re-synthesized. Every detector found them harder than the text-to-image samples (30–43% versus 41–57% detected), and in the paired comparison the edited image scores above its own source in only 65–86% of the pairs. This is the case that matters most in practice, because “AI-enhanced” photographs are far more common on social media than fully generated scenes, and the case our training data covers least: Community Forensics contains generated images, not edited photographs.

The role of the real-image source. A result that we did not expect at the start is how much the real images matter. The false alarm rate of ResNet-50 rose from 6% on Community Forensics real images to 28% on COCO photographs, with no change in the detector. A detector that is evaluated only against the real images of its own training distribution will look better than it is. We therefore think that a fair evaluation needs a second, independent source of real photographs, and that the normalization steps described in Section 3.3 (one file format, one resizing pipeline) are necessary but not sufficient: they remove the trivial shortcuts, but the detectors still partly learn what "real" looks like in one particular dataset.

Limitations. The study is small, which limits the conclusions. First, with 12 generators, 3 of them held out, and one external generator, we cannot separate the effect of the generator family from the effect of the individual generator; the drop on LFM and VQ-Diffusion may be a property of these two models rather than of non-GAN generators in general. Second, the training set has 1,836 images and 125 images per generator, and each seen generator has only about 19 test images, so the per-generator rates in Table 4 are coarse even when averaged over seeds; the bootstrap intervals in Table 3 show how much of a difference can be noise. Third, the Community Forensics subset we could process on free hardware is dominated by GANs and contains no modern latent-diffusion model, because the generators with small parquet files are mostly the hand-run models of the Manual subset; the training set is therefore not representative of the current generator population, which is part of why the transfer to SDXL-Turbo is hard. Fourth, the near-duplicate check uses a 64-bit perceptual hash, which finds near-copies but not every pair of images that share a latent code. Finally, we did not test robustness to re-compression or resizing after generation, which is known to reduce detection accuracy [30], and we did not adapt any detector to the new domain beyond the threshold analysis of Table 5.

7 Conclusion

We reviewed the main approaches to the detection of deepfakes and AI-generated images and why detectors fail on new generators, and we tested three baseline detectors on a dataset that combines a subset of Community Forensics with 2,000 images collected by ourselves, built so that the three evaluation settings differ only in the property we want to measure: new images, new generators and a new data source. The results, averaged over three seeds and reported with confidence intervals, show a three-level drop that is clearly visible even in a study of this size. Across generators of the benchmark, a linear classifier on frozen CLIP features generalizes with little loss (AUC 1.00 on three held-out generators, where a fine-tuned ResNet-50 drops to 0.90), in line with the universal-detector results of [4, 24], with the caveat that three held-out generators cannot establish this for non-GAN generators in general. Across data sources, the accuracy of all three detectors at a fixed threshold of 0.5 falls to 0.57–0.70 on our own COCO + SDXL-Turbo set; for the CLIP-based detector most of this drop is a calibration problem (AUC 0.85, accuracy 0.77 after choosing the threshold on 200 images of the new domain), while for the two networks it is mainly a loss of ranking ability. Partially re-synthesized photographs are detected in only 30–43% of cases by any detector. Beyond the numbers, three simple steps are necessary before a detection result can be trusted: a common file format and resizing pipeline for all images, a check for near-duplicates across the split boundaries, and a second, independent source of real photographs to measure false alarms honestly. We recommend that student projects in this area include all three and always report accuracy on a new domain together with AUC and a threshold analysis.

Code, split files, configuration and the predictions of every detector:
https://github.com/a33558771-prog/deepfake-generalization

References

[1] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images", Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV), 2019, pp. 1–11.

[2] L. Verdoliva, "Media forensics and deepfakes: An overview", IEEE Journal of Selected Topics in Signal Processing, 14(5):910–932, 2020.

[3] S.-Y. Wang, O. Wang, R. Zhang, A. Owens and A. A. Efros, "CNN-generated images are surprisingly easy to spot… for now", Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 8695–8704.

[4] U. Ojha, Y. Li and Y. J. Lee, "Towards universal fake image detectors that generalize across generative models", Proc. CVPR, 2023, pp. 24480–24489.

[5] R. Corvi, D. Cozzolino, G. Zingarini, G. Poggi, K. Nagano and L. Verdoliva, "On the detection of synthetic images generated by diffusion models", Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP), 2023.

[6] J. Park and A. Owens, “Community Forensics: Using thousands of generators to train fake image detectors”, Proc. CVPR, 2025. arXiv:2411.04125.

[7] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár and C. L. Zitnick, “Microsoft COCO: Common objects in context”, Proc. European Conf. on Computer Vision (ECCV), 2014, pp. 740–755.

[8] A. Sauer, D. Lorenz, A. Blattmann and R. Rombach, “Adversarial diffusion distillation”, arXiv:2311.17042, 2023.

[9] B. Dolhansky, J. Bitton, B. Pflaum, J. Lu, R. Howes, M. Wang and C. C. Ferrer, “The DeepFake Detection Challenge (DFDC) dataset”, arXiv:2006.07397, 2020.

[10] Y. Li, X. Yang, P. Sun, H. Qi and S. Lyu, “Celeb-DF: A large-scale challenging dataset for deepfake forensics”, Proc. CVPR, 2020, pp. 3207–3216.

[11] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio, “Generative adversarial nets”, Advances in Neural Information Processing Systems (NeurIPS), 2014, pp. 2672–2680.

[12] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen and T. Aila, “Analyzing and improving the image quality of StyleGAN”, Proc. CVPR, 2020, pp. 8110–8119.

[13] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen and T. Aila, “Alias-free generative adversarial networks”, NeurIPS, 2021.

[14] A. Brock, J. Donahue and K. Simonyan, “Large scale GAN training for high fidelity natural image synthesis”, Proc. Int. Conf. on Learning Representations (ICLR), 2019.

[15] J. Ho, A. Jain and P. Abbeel, “Denoising diffusion probabilistic models”, NeurIPS, 2020.

[16] R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, “High-resolution image synthesis with latent diffusion models”, Proc. CVPR, 2022, pp. 10684–10695.

[17] A. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. McGrew, I. Sutskever and M. Chen, “GLIDE: Towards photorealistic image generation and editing with text-guided diffusion models”, Proc. Int. Conf. on Machine Learning (ICML), 2022.

[18] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna and R. Rombach, “SDXL: Improving latent diffusion models for high-resolution image synthesis”, arXiv:2307.01952, 2023.

[19] F. Marra, D. Gragnaniello, L. Verdoliva and G. Poggi, “Do GANs leave artificial fingerprints?”, Proc. IEEE Conf. on Multimedia Information Processing and Retrieval (MIPR), 2019, pp. 506–511.

[20] X. Zhang, S. Karaman and S.-F. Chang, “Detecting and simulating artifacts in GAN fake images”, Proc. IEEE Int. Workshop on Information Forensics and Security (WIFS), 2019.

[21] R. Durall, M. Keuper and J. Keuper, “Watch your up-convolution: CNN based generative deep neural networks are failing to reproduce spectral distributions”, Proc. CVPR, 2020, pp. 7890–7899.

[22] J. Frank, T. Eisenhofer, L. Schönherr, A. Fischer, D. Kolossa and T. Holz, “Leveraging frequency analysis for deep fake image recognition”, Proc. ICML, 2020.

[23] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger and I. Sutskever, “Learning transferable visual models from natural language supervision”, Proc. ICML, 2021.

[24] D. Cozzolino, G. Poggi, R. Corvi, M. Nießner and L. Verdoliva, “Raising the bar of AI-generated image detection with CLIP”, Proc. CVPR Workshops, 2024.

[25] Z. Wang, J. Bao, W. Zhou, W. Wang, H. Hu, H. Chen and H. Li, “DIRE for diffusion-generated image detection”, Proc. ICCV, 2023.

[26] C. Tan, Y. Zhao, S. Wei, G. Gu, P. Liu and Y. Wei, “Rethinking the up-sampling operations in CNN-based generative network for generalizable deepfake detection”, Proc. CVPR, 2024.

[27] M. Zhu, H. Chen, Q. Yan, X. Huang, G. Lin, W. Li, Z. Tu, H. Hu, J. Hu and Y. Wang, “GenImage: A million-scale benchmark for detecting AI-generated image”, NeurIPS Datasets and Benchmarks, 2023.

[28] S. Yan, O. Li, J. Cai, Y. Hao, X. Jiang, Y. Hu and W. Xie, “A sanity check for AI-generated image detection”, Proc. ICLR, 2025. arXiv:2406.19435.

[29] L. Chai, D. Bau, S.-N. Lim and P. Isola, “What makes fake images detectable? Understanding properties that generalize”, Proc. ECCV, 2020.

[30] D. Gragnaniello, D. Cozzolino, F. Marra, G. Poggi and L. Verdoliva, “Are GAN generated images easy to detect? A critical analysis of the state-of-the-art”, Proc. IEEE Int. Conf. on Multimedia and Expo (ICME), 2021.

[31] M. Kang, J.-Y. Zhu, R. Zhang, J. Park, E. Shechtman, S. Paris and T. Park, “Scaling up GANs for text-to-image synthesis”, Proc. CVPR, 2023.

[32] A. Sauer, K. Chitta, J. Müller and A. Geiger, “Projected GANs converge faster”, NeurIPS, 2021.

[33] A. Sauer, K. Schwarz and A. Geiger, “StyleGAN-XL: Scaling StyleGAN to large diverse datasets”, Proc. ACM SIGGRAPH, 2022.

[34] Y. Takida, M. Imaizumi, T. Shibuya, C.-H. Lai, T. Uesaka, N. Murata and Y. Mitsufuji, “SAN: Inducing metrizability of GAN with discriminative normalized linear layer”, Proc. ICLR, 2024.

[35] P. Esser, R. Rombach and B. Ommer, “Taming transformers for high-resolution image synthesis”, Proc. CVPR, 2021, pp. 12873–12883.

[36] D. A. Hudson and C. L. Zitnick, “Generative adversarial transformers”, Proc. ICML, 2021.

[37] Q. Dao, H. Phung, B. Nguyen and A. Tran, “Flow matching in latent space”, arXiv:2307.08698, 2023.

[38] S. Gu, D. Chen, J. Bao, F. Wen, B. Zhang, D. Chen, L. Yuan and B. Guo, “Vector quantized diffusion model for text-to-image synthesis”, Proc. CVPR, 2022, pp. 10696–10706.

[39] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition”, Proc. CVPR, 2016, pp. 770–778.